

LICENSING OFFICER

Review queue

LICENSING OFFICER

Review queue

3 need review · 0 waiting on operators · 4 decided

Refreshed just now · updates every 30 s



PermitFlow

PermitFlow: how it is built

Scope, architecture, the decisions that make the brief's guarantees true, the pipeline, the evidence, and the gaps. Every claim names its file.

mitFlow

REFERENCE

BUSINESS · APPLICANT

STATUS · CHECKS

PF-2026-001009

Kopi & Kaya Toast House Pte. Ltd.

Revision 1 · 2 open

Tan Wei Ling · 10 Jalan Besar #01-12 · just now

Continue review →

● Under Review

● 2 to check

just now

Submitted 19 Sep 2026

PF-2026-001001

Scenario Kopi House Pte. Ltd.

Revision 1

Tan Wei Ling · 10 Jalan Besar #01-12 · 14 h ago

Continue review →

● Under Review

● 4 to check

14 h ago

Submitted 19 Sep 2026

PF-2026-001000

Scenario Kopi House Pte. Ltd.

Revision 1

Tan Wei Ling · 10 Jalan Besar #01-12 · 19 h ago

Continue review →

● Under Review

● 4 to check

19 h ago

Submitted 18 Sep 2026

Flow: how it is built

re, the decisions that make the brief's guarantees true, the pipeline, the

e gaps. Every claim names its file.

Three use cases, one status table, five things being evaluated.

UC1 **Operator submission and resubmission**

● Built

Form with sections, drag-and-drop uploads, live AI status per document, progress, feedback on top, edit only flagged parts, unlimited rounds, history.

UC2 **Officer review and feedback**

● Built

Full submission, AI results, contextual feedback with templates, status change notifies, changed sections highlighted, compare revisions, resolution tracked, audit trail.

UC3 **On-site assessment and post-site clarification**

● Deferred

The three post-site statuses and their transitions exist and are tested; the checklist model and screens are not built. Recorded in SCOPE.md with what production needs.

STATUS TABLE

Status table: 12 internal statuses with a different label per role. Implemented as data in domain/labels.py, verbatim from the brief, plus draft and withdrawn as written assumptions.

WHAT IS EVALUATED, AND WHERE IT IS IN THIS DECK

Scope judgement	slides 3 to 5
Production readiness	slides 12 to 17
AI tool usage	slides 18 to 21
Code quality	slides 6 to 11
Documentation	docs/README.md is the index; docs/11-reviews/ASSESSMENT_TRACEABILITY.md maps every brief line to code, test and evidence

One polished, tested, deployed vertical slice. Everything else written down with what production would need.

● MUST**20 of 20 built**

- Form, uploads with allowlist, magic bytes and 10 MB
- AI check per document with live status
- Explicit state machine, 14 statuses, role labels
- Feedback tied to a section or document, templates
- Targeted resubmission, immutable revisions, compare
- Audit trail, in-app notifications
- Server-side authorization, error envelope
- Tests per layer, CI, Railway, README, SCOPE.md

● SHOULD**6 of 7 done**

- Queue groups and search
- Re-run a check (operator and officer)
- Delete a document while in draft
- Compare any two revisions
- Evaluation set: 14 cases, mock gate, live nightly
- Request logging with request id
- S7 admin persona: deferred to v0.4.0

● COULD**5 of 8 done**

- Dashboard summary
- Withdraw with a reason (new terminal status)
- Licence certificate on approval, preview, PDF
- Owner's domain, two environments, TLS
- Legal, privacy and accessibility review
- Open: officer assignment, image OCR, bundle download

● DEFERRED**13 rows, grouped here**

- UC3 checklist (1.5 days estimated)
- Email delivery (mocked, in-app only)
- Worker queue for AI checks (in-process today)
- Object storage (volume today)
- Registration, reset, MFA, SSO
- Configurable forms, OCR, push, WAF, virus scan
- Session in sessionStorage, US-region providers, one replica: disclosed with severity

Where the brief was ambiguous, the assumption is written down and enforced by code.

01 **A draft state exists before Application Received**

Needed for save and return; never shown to officers; deletable outright until submitted (US-045). SCOPE.md assumption 2 and 14.

02 **Under Review starts on an explicit officer action**

So the actor lands in the audit trail. Start review is a transition row, not a side effect. Assumption 3.

03 **Food licensing; the operator applies for the business**

The brief names no licence: I chose Singapore's Food Establishment Licence (four sections, four documents an AI check can compare with the form). Operators are the business owner or an agent applying on its behalf; the account holder is the applicant, authority is not verified. Assumptions 1 and 10.

04 **Addressed is automatic, resolved is a decision**

The linked section or document changed in the next revision: addressed. The officer confirms: resolved. Assumption 7.

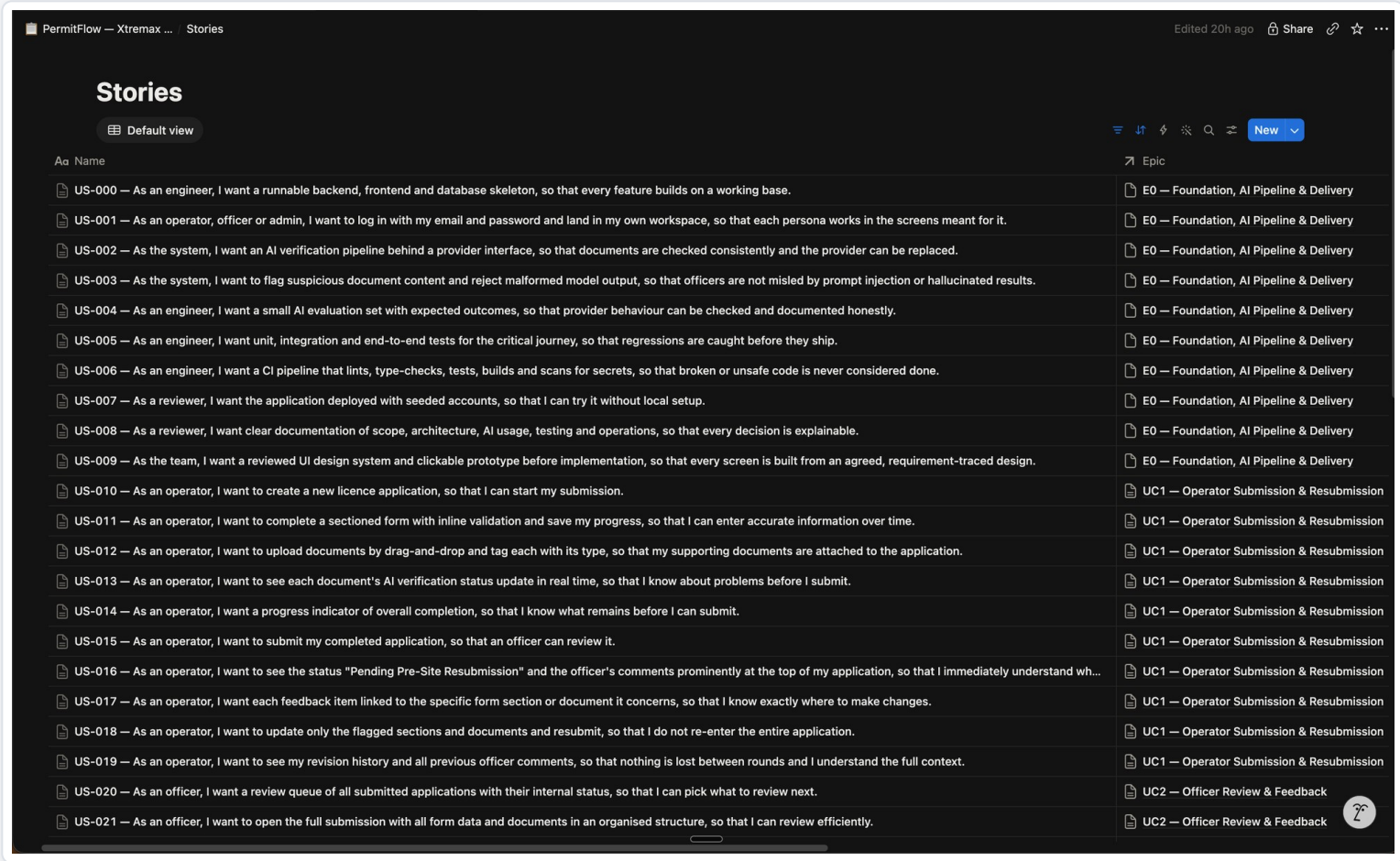
05 **Only flagged sections may change on resubmission: enforced**

The API returns 403 for any other change. A product decision open to reversal. Assumption 13, domain/resolution.py.

06 **Operators see Pending Approval as a waiting state**

The brief's own table gives that label; the officer-only label Route to Approval and the decision note never reach an operator. Assumption 17.

Three one-day sprints on a board that mirrors the repository, a definition of done per story, a close ritual per sprint.



Board and repository, one to one

74 stories in five epics on the board (11 deferred: use case 3 and the admin epic); USER_STORIES.md carries the same ids, titles and acceptance criteria; every branch name carries its story id. Design phase 17 Sep, Sprints 1 to 3 on 18 and 19 Sep.

Cadence

Planning, 15 minutes at the start of the day. Mid-sprint check at midday: cut or simplify then, not at 23:00. Review and close at the end of the day. Two stories in progress at most.

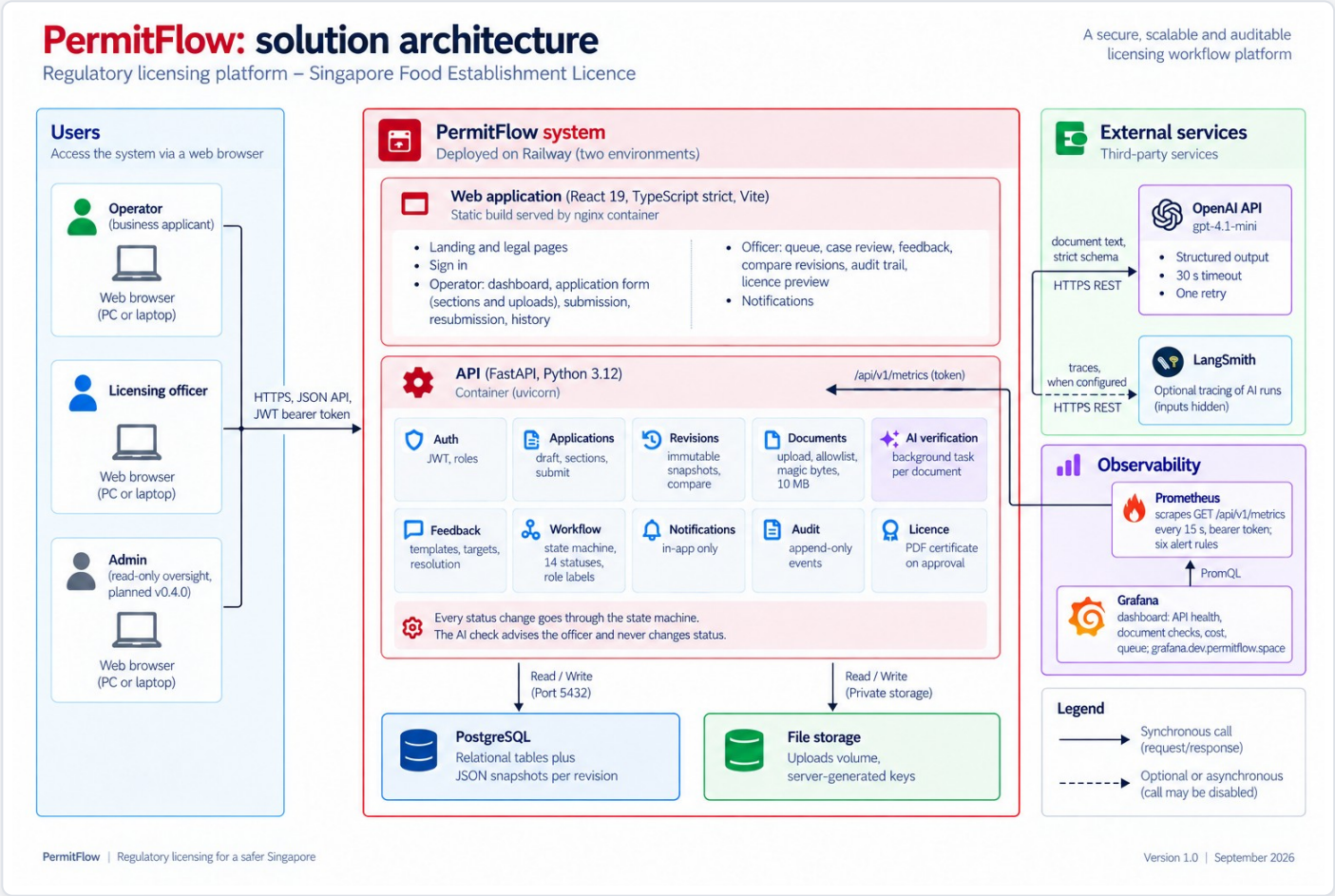
Definition of done, per story

Acceptance criteria met; server validation and authorization tested with the wrong role and owner; the standard error body; audit in the same transaction; no path from AI output to state; tests per layer green in CI; docs and the board updated.

Close ritual, run three times

Tests green, commits checked, board final, CHANGELOG entry with Shipped, Slipped and Retro, SCOPE.md re-read, commit docs: close sprint N. At the third close the admin epic was cut per the cut order, to v0.4.0.

A modular monolith: one API in strict layers, one web app, one database, an isolated AI module.



Layers, enforced

api, services, domain, repositories, models. domain is pure Python: no SQLAlchemy, no FastAPI, no I/O. tests/unit/test_layering.py fails the build if a layer imports downward the wrong way. ADR-001.

PostgreSQL only

Relations for users, applications, feedback and audit; every submitted revision an immutable JSON snapshot. Row locks and a version column for concurrency. ADR-002.

AI in-process, behind an interface

Background task per document, result polled every 2 s, stale runs marked failed on restart. VerificationProvider with OpenAI and a deterministic mock. ADR-004, the worker is next.

Stack and telemetry

FastAPI, SQLAlchemy 2, Pydantic v2, Alembic; React 19, TypeScript strict, Vite, TanStack Query, React Hook Form, Zod; pytest, vitest, Playwright (ADR-009). Prometheus scrapes GET /api/v1/metrics behind a token; Grafana shows one dashboard; six alert rules (US-077).

The workflow is a table, not a set of if statements.

CHOSE

The fourteen statuses and every allowed move as data: from, to, actor, guard, label. One function, `transition()`, consults it. `available_actions()` gives the UI its buttons with the reason a button is disabled.

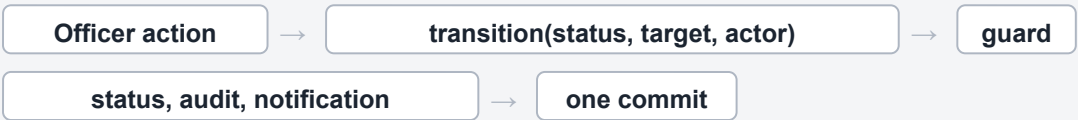
OVER

Status changes scattered through services, or a workflow library with its own model of the world.

BECAUSE

A table can be compared with the brief's status table line by line and tested exhaustively. Adding a status is a row. Reject is possible from every state the MVP can reach and withdraw from every post-submission state, so no case can get stuck; the three post-site states get reject when use case 3 ships.

THE PATH



EVIDENCE IN THE CODE

`domain/workflow.py`

16 explicit rows, plus reject from 7 states and withdraw from 10 (33 transitions); guards: `is_complete`, `open_feedback_count`, `has_changes_to_flagged_targets`

`domain/labels.py`

the brief's table verbatim: internal status, officer label, operator label, badge tone

`tests/unit/test_workflow.py`

every (source, target, actor) combination: $14 \times 14 \times 3 = 588$ cases; terminal states have no exits; guards

`POST /officer/applications/{id}/transition`

row lock, `expected_version`, 409 `invalid_transition` with the allowed list, audit and notification in the same transaction

TRADE-OFF

Guards are Python callables, so a guard that needs data the context does not carry means extending the context. Accepted: the context has three fields and has not grown since Day 1.

The table in `domain/workflow.py`: a row per allowed move, one function that consults it.

```
@dataclass(frozen=True)
class Transition:
    source: S
    target: S
    actor: Actor
    guard: Guard | None = None
    label: str = ""

TRANSITIONS: tuple[Transition, ...] = (
    Transition(S.DRAFT, S.APPLICATION_RECEIVED,
              Actor.OPERATOR, _needs_complete, "Submit"),
    Transition(S.APPLICATION_RECEIVED, S.UNDER_REVIEW,
              Actor.OFFICER, None, "Start review"),
    ...
    Transition(S.PENDING_APPROVAL, S.APPROVED,
              Actor.OFFICER, None, "Approve"),
    Transition(S.PENDING_APPROVAL, S.UNDER_REVIEW,
              Actor.OFFICER, None, "Return to review"),
)
TRANSITIONS += tuple(
    Transition(src, S.REJECTED, Actor.OFFICER, _needs_note, "Reject")
    for src in _REJECT_SOURCES
)
```

Data, not branches

A frozen dataclass per row; the tuple is the whole policy. `transition()` looks a row up, runs its guard, and returns the new status or a typed error (invalid, forbidden, guard).

Guards are named

`_needs_complete`, `_needs_note`, open feedback counts: small callables over a three-field context, listed in the evidence on the previous slide.

The sweep

`test_workflow.py` builds VALID from this tuple and iterates 14 x 14 x 3 combinations, asserting the kind of refusal, not only that it refused.

Operators never see internal status codes, because the API cannot send them.

CHOSE

Role checked per router; ownership enforced in the one function that loads an application (another owner's case is a 404, not a 403); a separate response model per role, so the operator view has no field for the internal status or the officer label.

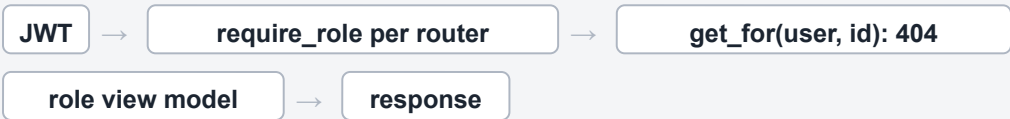
OVER

Checking roles in the UI, or one shared response shape with fields hidden client-side.

BECAUSE

The brief's constraint, operators cannot see the approval stage, becomes a property of the types and the tests, not a hope. A leaked field would be a schema change, which a reviewer sees in the diff.

THE PATH



EVIDENCE IN THE CODE

api/deps.py

get_current_user (JWT, active user), require_role('officer'), require_role('admin')

repositories/applications.py

get_for(user, id) applies the operator_id filter; NotFound for another owner

schemas/applications.py vs schemas/officer.py

operator view: operator label, released feedback without author, no internal status, no decision note before the outcome

tests/integration

an authorization test for every endpoint: wrong role 403, wrong owner 404, cross-application ids 404, admin read-only

TRADE-OFF

Two response models to keep in step when a field is added. Accepted: the layering test and the API table in ARCHITECTURE.md make a missed field visible.

The AI check advises. Strict schemas and deterministic rules decide what an answer is allowed to mean.

CHOSE

A provider interface with two implementations (OpenAI structured output, deterministic mock). A pinned wire schema, then a strict domain model (extra forbidden, confidence 0 to 1, status must agree with the issue list), then rules after the model: confidence under 0.6 or an injection phrase forces needs_review. The result is stored on the run and shown to the officer. It never touches status, feedback or editability.

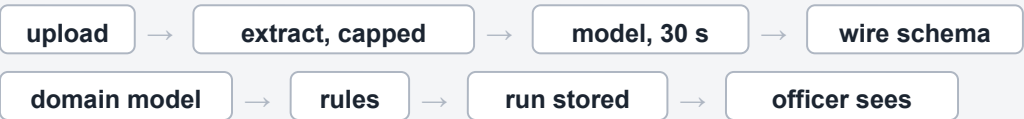
OVER

Letting the model's answer drive the workflow, or free-text answers parsed by hand.

BECAUSE

The brief says AI assists and the officer decides. With strict schemas a bad answer becomes a flagged answer, never a state change. Over quota or on failure the run is stored unavailable and the application proceeds.

THE PATH



EVIDENCE IN THE CODE

`infra/ai/base.py`, `openai_provider.py`, `mock.py`

VerificationProvider; prompt_version 2026-09-19.3 stamped on every run; 30 s timeout, one retry

`domain/verification_rules.py`

VerificationResult (extra=forbid), apply_rules(threshold, injection phrases), find_injection_phrases

`services/verification.py`

the only caller of infra.ai; own session and transaction; text extraction capped (30 pages, 20 000 chars, 10 s); never raises out of the task

`evals/`: 14 golden cases, 2 injection cases, 21 fairness runs (18 name-swapped)

mock gate on every push, live nightly, LangSmith experiment per run; docs/07-ai/AI_ASSURANCE.md

TRADE-OFF

Confidence is the model's own estimate and is uncalibrated; the injection heuristic is English-only phrases. Both recorded as next steps with the tools to fix them.

Nothing is lost between rounds: every submission is a frozen snapshot, and every change writes its audit row in the same transaction.

CHOSE

Each submit or resubmit stores Revision N: the form as JSON plus the document ids. The comparison between revisions is computed at read time from the snapshots. Audit events, notifications and the licence on approval are written in the same database transaction as the change; there is no update or delete path for audit rows.

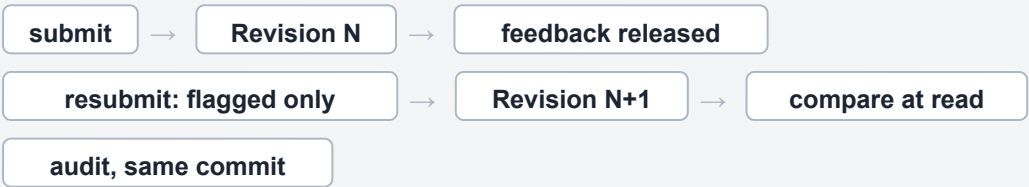
OVER

Editing one mutable record, storing diffs instead of snapshots, or a separate logging pipeline and database triggers.

BECAUSE

Never lost between rounds and complete audit trail become true by construction. The officer's compare view cannot disagree with what was submitted. A change without its audit row cannot exist, and vice versa.

THE PATH



EVIDENCE IN THE CODE

models: application_revisions, audit_events
immutable rows; raised_in_revision on feedback; sha256 on documents so an identical re-upload is no change

domain/diff.py, services/compare.py
field-level form diff and document add, remove, replace; GET /applications/{id}/compare?from=n&to=m

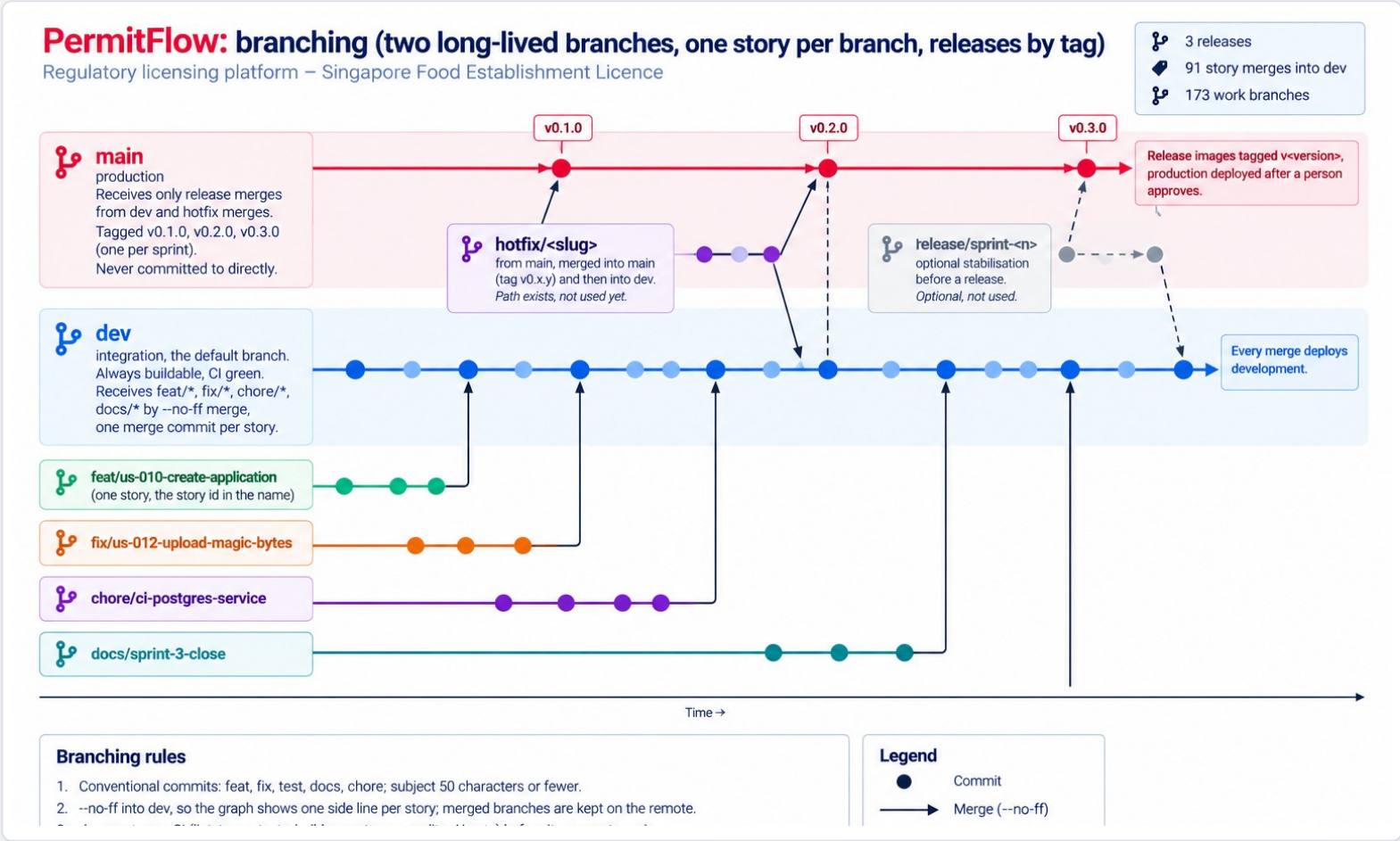
services/resubmission.py
SELECT FOR UPDATE, only flagged targets may change (403), at least one flagged target changed (422), Revision N+1, feedback to addressed, audit, notify, one commit

GET /officer/applications/{id}/audit
actor, role, plain-language summary from domain/audit_labels.py, payload, chronological; 752 backend test cases include every event type

TRADE-OFF

Snapshots grow with rounds; at this form size a revision is a few kilobytes. A separate audit store would be the answer at a scale this product is not at.

Two long-lived branches, one story per branch, releases by tag.



main and dev

main is production and receives only release merges from dev (or a hotfix); tagged v0.1.0, v0.2.0, v0.3.0, one per sprint. dev is integration and the default branch: always buildable, CI green, every merge deploys development.

One story per branch

feat/us-<id>-<slug>, fix/*, chore/*, docs/*, each from dev and back into dev with --no-ff, so the graph shows one side line per story and the branch name maps to the board. 153 merge commits on dev and 86 branches still on the remote (git log --merges dev, git branch -r), on 20 Sep.

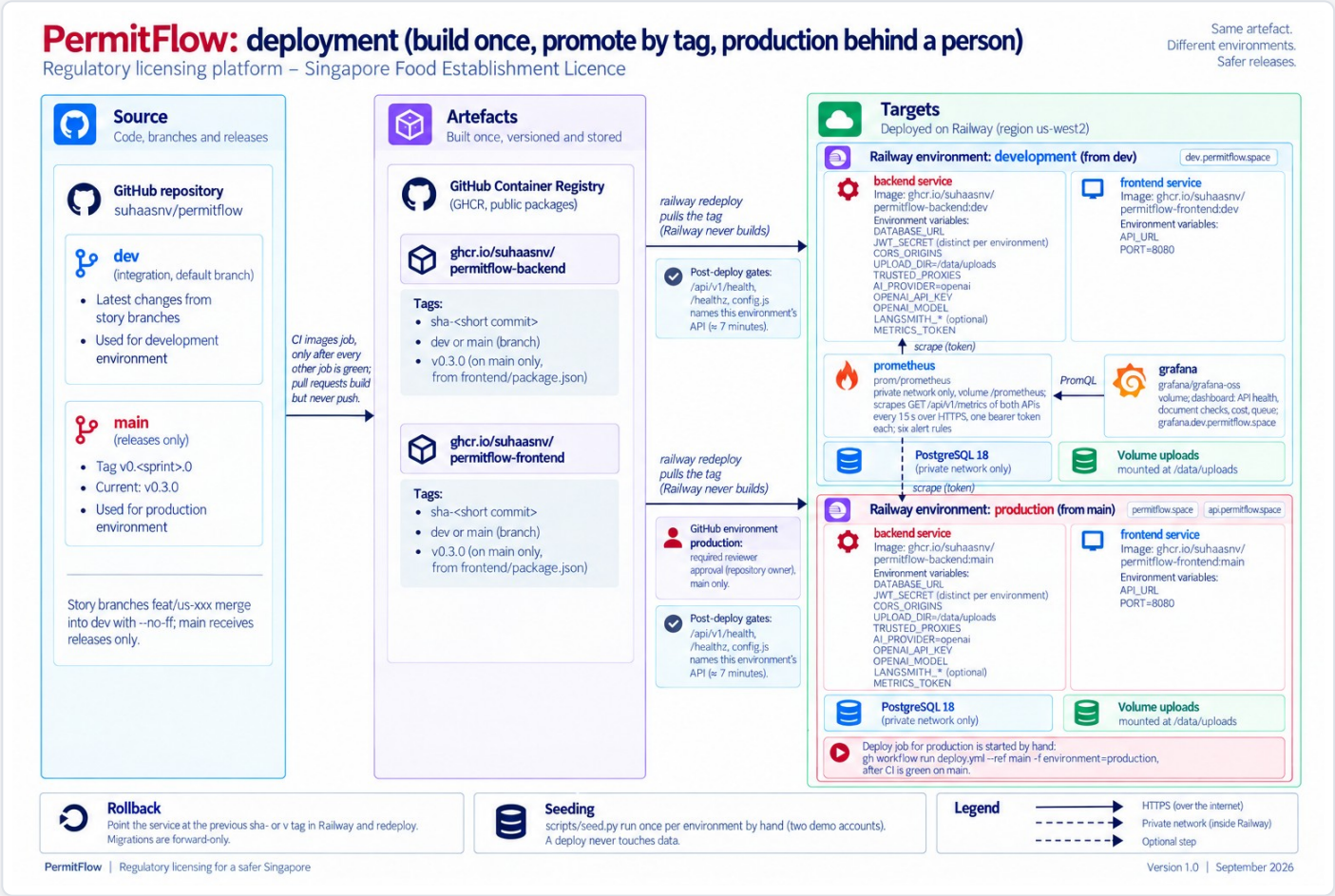
Release and hotfix

A release is the dev pull request merged into main, then a git tag on that commit; only the tag's CI run writes the v0.x.0 images. Production is then pinned to that sha- image and deployed with approval. A hotfix branches from main and merges into main, then dev.

Rules

main is protected: changes arrive by pull request from dev with seven green checks, no force pushes, the rule applies to the owner. dev is protected against force pushes. Conventional commits; history never rewritten. docs/09-operations/BRANCHING.md.

Build once, promote by tag, production behind a person.



Images built once

CI builds both images after every other job is green and pushes them to GHCR as sha-<commit> and the branch name. A release tag (v0.3.0) is written only by the CI run for that git tag and must match package.json, so it can never be overwritten. Railway never builds.

Two environments, plus the monitor

development from dev, production from main; own PostgreSQL 18, volume, secrets, tokens, domains. Development also runs prometheus (private, scrapes both APIs, one token each) and grafana at grafana.dev.permitflow.space, from public images.

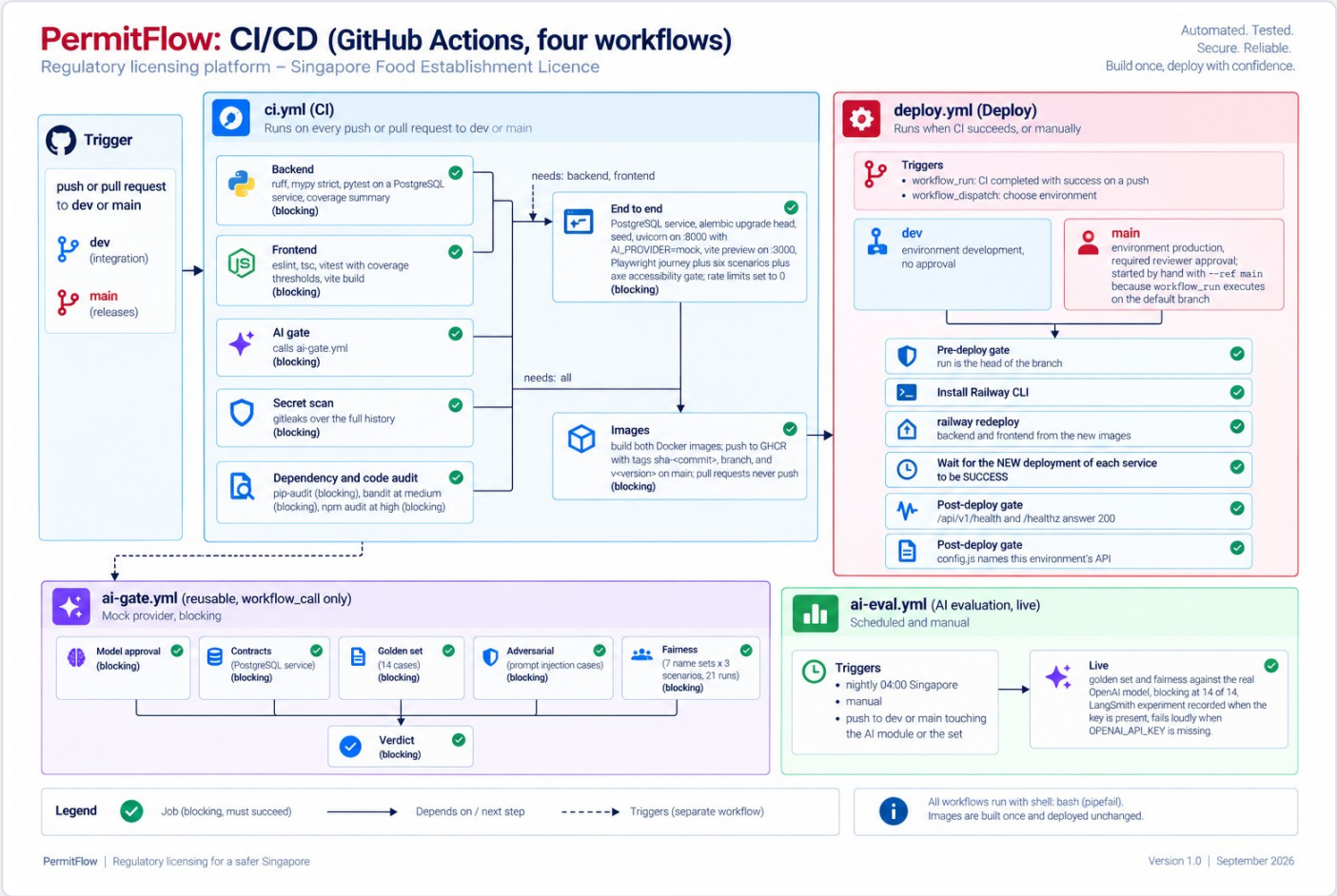
Gates

Production is pinned to a release sha-<commit> image, changed by hand at a release. The deploy job records the live deployment id, waits for a different id to reach SUCCESS, then checks /api/v1/health, /healthz and config.js. Production waits for the owner's approval in GitHub.

Known limit

A workflow_run job executes on the default branch, dev, which the production environment does not admit, so production is deployed by hand: gh workflow run deploy.yml --ref main, with the approval and the gates. Documented in OPERATIONS.md.

Seven CI jobs, an AI gate with six, a live evaluation, and a deploy that waits for health.



Everything blocks

Backend (ruff, mypy strict, 763 pytest cases on PostgreSQL), Frontend (eslint, tsc, 158 vitest with thresholds, build), gitleaks, pip-audit, bandit, npm audit, End to end (the whole stack in the runner, Playwright), Images. No job is advisory.

AI gate on every push

Model approval, contracts on a real database, 14 golden cases, adversarial injection cases, 21 fairness runs (18 name-swapped), verdict. On the mock provider so it is deterministic and free.

Live evaluation

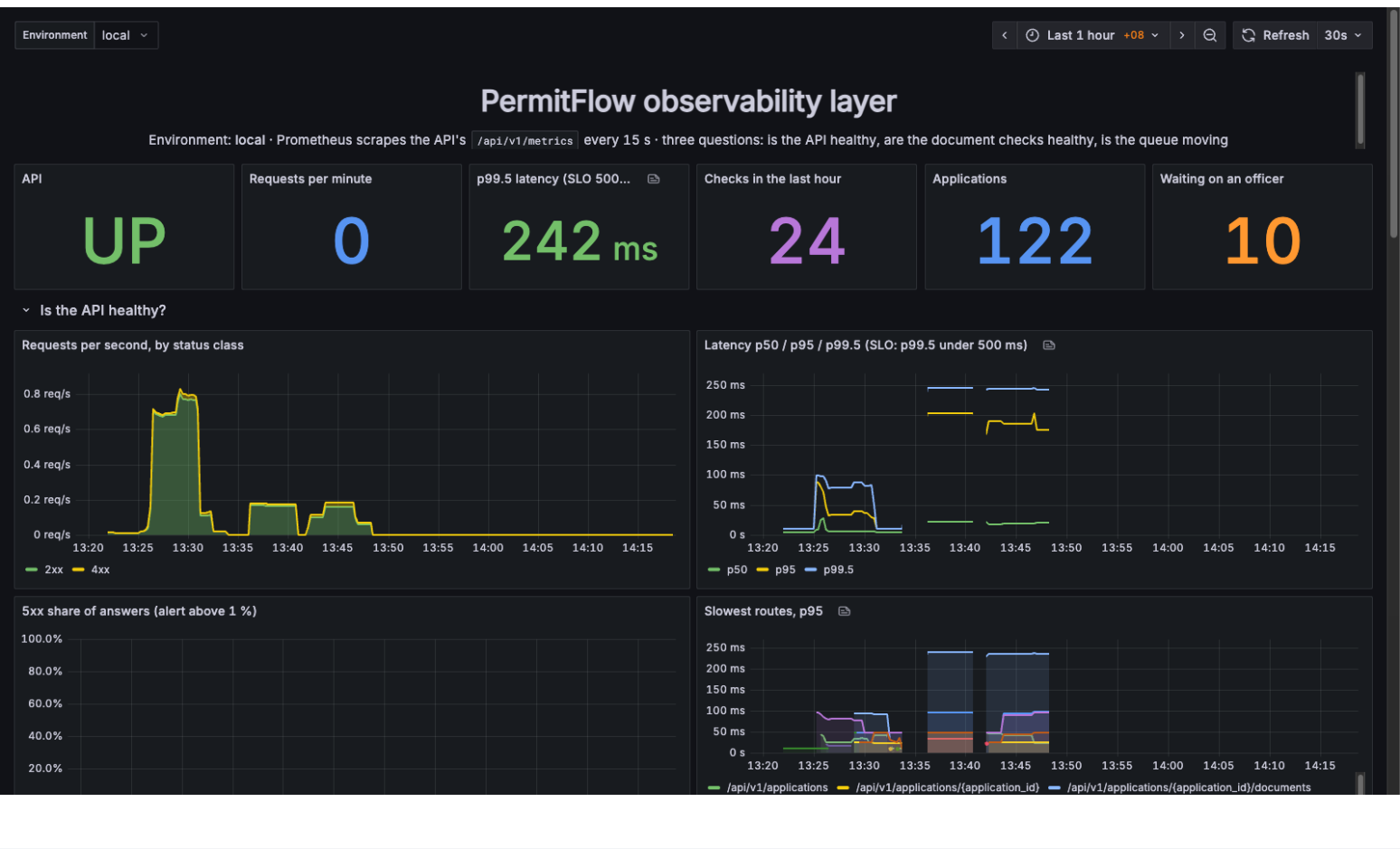
Nightly and on AI-path pushes against the real OpenAI model: blocking at 14 of 14, each run recorded as a LangSmith experiment. Fails loudly when the key is missing.

Reproducible

The end-to-end job runs migrations, seed, uvicorn and vite preview inside the runner, with rate limits set to 0 and the mock provider. The same commands run locally.

.github/workflows: ci.yml, ai-gate.yml (workflow_call), ai-eval.yml, deploy.yml · shell: bash with pipefail everywhere

Prometheus scrapes the API behind a token; one Grafana dashboard answers three questions and a fourth about cost.



What is counted

Requests by route and status with latency; limiter and quota refusals; checks by outcome and provider with latency; transitions; applications by status; the tokens each check bills. Labels: templates and enums, never an id.

The endpoint

GET /api/v1/metrics: off unless METRICS_TOKEN is set, 401 without the bearer, exempt from the limiter, never counts itself. T23.

Dashboard, alerts, Telegram

API health, check health, cost at list price, queue. Six rules on the objective (99.5 % under 500 ms, checks within 3 min, 5xx under 1 %) fire to Telegram with an hourly digest per environment; a bot answers /status, /cost, /queue.

Where it runs

One Compose profile locally; on Railway three services from public images: prometheus (private, both environments), grafana at grafana.dev.permitflow.space, the Telegram bot.

The rules fire to Telegram, resolve themselves, and a bot answers the question an officer would ask.

RESOLVED · PermitFlow API down
Environment **development** · severity **critical**
The API's metrics endpoint has not answered for two minutes.
Value now **1**
Alerting <https://grafana.dev.permitflow.space/alerting/list> 15:00

PermitFlow · development

Development · API up
Requests **3 / min** · p99.5 **240 ms** · 5xx **0.0 %**
Checks (1 h) **6** · spend **USD 0.002**
Applications **13** · officer's turn **1** · operator's turn **0**

Dashboard

<https://grafana.dev.permitflow.space/d/permitflow/permitflow>

15:04

What arrived

An alert when the development target went dark before the endpoint shipped, its RESOLVED the minute the deploy landed, an hourly digest with a section per environment, and /dev answered from Prometheus: API up, 3 requests a minute, p99.5 240 ms, 6 checks for USD 0.002, 13 applications.

How

Grafana-managed rules mirror the six Prometheus rules, one alert per environment, with a Telegram contact point; the digest is an always-firing rule repeated hourly; the bot is 200 lines of standard-library Python that only reads Prometheus and only answers one chat.

Provisioned, not clicked

Rules, contact point, policy and message template are generated by build_alerting.py and provisioned at start, only when the bot token and chat id are set; an instance without them removes its own alerting.

Honest limits

No acknowledgement, no escalation, no silence window, nobody on call. Aggregate numbers only: no id, no name, no document text ever leaves for Telegram.

Checked by tests, gates and a written record.

763 backend test cases, 95 % statements **158** frontend tests, 81 % **8** Playwright specs, axe over 23 states **23** threats, each with a control **24** readiness rows with severity

- ✓ **Input validation** Pydantic 422 with per-field details; upload allowlist, size streamed to 10 MB, magic bytes; zod schemas built from the server's form schema.
- ✓ **Error envelope** { error: { code, message, details } } on every route, including 401, 403, 422, 429 and 500 with a request id and no stack trace.
- ✓ **Concurrency** SELECT FOR UPDATE on the application; expected_version on transitions (409 version_conflict); stale AI runs recovered on restart.
- ✓ **Abuse limits** 240 requests and 20 sign-ins per client per minute with Retry-After; 20 open drafts; 60 checks per applicant and 1 000 per day, over quota stored unavailable.
- ✓ **Headers and secrets** CSP, HSTS, COOP, Permissions-Policy on both tiers; Argon2, JWT 8 h re-validated per request; gitleaks over the history; .env.example documents every variable.
- ✓ **Accessibility and legal** axe gate in CI over 23 screen states; labels never colour alone; policy pages and demonstration notices; fonts self-hosted.
- ✓ **Observability** GET /api/v1/metrics behind a bearer token: requests, latency, refusals, check outcomes and cost, transitions, queue; Prometheus with six alert rules; one Grafana dashboard; alerts, an hourly digest and a command bot on Telegram; local and on Railway (US-077).

Threat model

docs/06-security/THREAT_MODEL.md: T1 to T23 plus T1a, STRIDE per asset, the control for each, what is accepted for a demonstration and why.

Readiness review

docs/11-reviews/PRODUCTION_READINESS_REVIEW.md: 24 rows, severity, what mitigates today, what production needs.

Security hardening

docs/06-security/SECURITY_REVIEW.md: four attack scenarios walked through against the code, with the fix or the accepted risk.

UAT

docs/10-uat/UAT_PLAN.md: twelve scripts, executed on development and again on production after v0.3.0.

AI wrote most of the code inside rules that are checked in. Every decision and every check is mine.

Models and tools	Claude Code in the Cursor terminal (Opus 5, some sessions Fable 5.1); Cursor to read every diff; Wispr Flow for the long briefs; Claude in Chrome for run-throughs and dashboards, my own sign-in, no secret through the agent.
Share of the work	Nearly all application code and most documents generated by the agent; every decision, prompt, review, merge and push mine. OpenAI only inside the product.
Workflow first	Requirements with ids, SCOPE.md, solutioning, a 23-artboard prototype, three one-day sprints on the board, one branch per story, CI/CD and two environments from day 3.
Standing context	Two instruction files in every session; the project one is checked in as CLAUDE.md (two slides on). The brief's five context types, each with an example in AI_USAGE.md.
How output was checked	Full suites, ruff, mypy strict, tsc strict on every story; browser checks at three widths; subagent reviews that had to reproduce a finding; docs checked against code at each close.

Six things the AI got wrong, what caught each one, and what changed.

01 **Invented enum values, and an expired certificate read as valid**

First live run of the verifier. Caught by reading the run. Fix: the vocabulary pinned in the wire schema, today's date in the prompt; the harness now gates every prompt change.

02 **A scope-creeping story and a red landing band**

Proposed unasked. Caught at review. Both rejected and recorded in the story notes; SCOPE.md changes only by decision.

03 **A transition the state machine did not allow**

An approval-stage option offered without checking the table. Caught by the sweep. It exposed a real gap: Return to review was added as a row.

04 **A scratch file swept into a commit**

A subagent's test file caught by git add -A while it was still running. Caught in the diff. Staging became explicit from then on.

05 **Licence dates computed in UTC**

A day off in Singapore. Caught in the rendered PDF. Fixed to Asia/Singapore with a test.

06 **A harness fault blamed on the model**

The first fairness run reported two name-swapped cases as differing; the harness had left the baseline email in the form. Caught by reading the traces. Twenty-one of twenty-one after the fix, recorded as such.

CLAUDE.md is checked in: the standing rules every session read first, and the checklist run after every story.

Sources of truth	SCOPE.md, REQUIREMENTS.md with ids, USER_STORIES.md one to one with the board, the architecture documents, the threat model. Code and a document disagree: fix one in the same change.
Personas and engineering rules	Three roles; server-side authorization; operators never receive internal codes; strict layering; every status change through workflow.py; the AI never mutates state; immutable revisions; same-transaction audit.
Notion board, same turn	In progress when a story starts, Done only on its definition of done, slipped with a note; new or renamed stories mirrored in the same turn.
After every story, the checklist	Tests per layer; env vars to .env.example, README, OPERATIONS; schema changes to a migration and DOMAIN_MODEL; endpoints to the API table plus an authorization test; ADRs; SCOPE.md at once; prompt changes to docs/07-ai.
Every hat, every day	One developer, so the agent carries PM, architect, QA, security, DevOps, writer and reviewer duties per story; sprint close daily; a handover line if a session stops mid-story; no push without my yes.

Two decks and three videos, made by a second session from the repository, with the same rules.

What was made	Launch video 70.7 s, narrated walkthrough 4 min 36 s, technical video 4 min, all 1920 x 1080 at 60 fps with subtitles; the pitch deck (15 slides) and this deck (23 slides). All in docs/14-debrief/.
How	A second Claude Code session after the code freeze, with the video skills installed for the workflow; Playwright footage from the development environment at 4K, a synthetic voice, catalogue music under it, rendered at 60 fps.
Guardrails	Plans reviewed by four persona subagents through fourteen revisions before any build; the code, not the design documents, as the source of truth for every label; a leak sweep of every recording; production never touched.
Diagrams	Generated by ChatGPT from prompts written against the workflows, Dockerfiles and operations documents, each with an acceptance list; rated against the code before use, two rejected for invented components.
Decks	One content.json per deck through a pptxgenjs generator; facts pulled from the repository; a cold hiring-manager review on 20 Sep corrected five counts and one claim before this version.

Known gaps first, then what production would need.

- 01

Use case 3
The checklist model, the officer capture screen with draft save, per-item clarification and the operator's targeted response. The statuses and transitions exist. About 1.5 days.
- 03

Object storage and backups
S3-compatible bucket with signed URLs and a virus scan; a file backup and a restore drill; a retention policy for documents and traces.
- 05

AI assurance
A labelled set from officer overrides; promptfoo or PyRIT red-team beyond two injection cases; Project Moonshot for Singapore evidence; calibrated confidence; in-region tracing; a multilingual injection classifier.
- 07

Delivery
Staging with a copy of production data for migration rehearsal; two replicas; Semgrep and CodeQL; Trivy with an SBOM. Observability, second half: alerts routed to a channel (Alertmanager), nginx and Postgres exporters, one Prometheus per environment, a runbook per alert.

- 02

A worker for the AI checks
Redis and a worker, or a Postgres SKIP LOCKED queue, so checks survive deploys and scale apart from the API. ADR-004 has one call site to change.
- 04

Session and edge
httpOnly cookie with CSRF protection instead of sessionStorage; CSP nonces; the rate windows in Redis or at the edge.
- 06

Admin epic, v0.4.0
Overview with AI health and quota counters, audit feed, read-only case view, user management. Role and routes reserved, screens not built.
- 08

Product
Email delivery, officer assignment, more licence types via a configurable form schema, a public verification page for certificates.



Every decision has a file. Every claim has a test.



Thank you. Questions on any file, any decision, any gap.

permitflow.space
github.com/SuhaasNv/permitflow